Lazy Loading and N+1 Problem

JPA / Hibernate - Chapter 4.5

Beginner-Friendly Guide with Examples and Diagrams

Eager vs Lazy Loading	FetchType.EAGER and FetchType.LAZY explained
The N+1 Problem	Why 1 + N queries can destroy performance
Fix 1: JOIN FETCH	Solve N+1 with a single JPQL JOIN
Fix 2: @BatchSize	Hibernate batch-loads associations
Fix 3: @EntityGraph	JPA standard for fetch plans
LazyInitializationException	Cause and fix in detail
When to Use EAGER vs LAZY	Decision guide for real projects

1 Eager vs Lazy Loading

FetchType.EAGER vs FetchType.LAZY

When JPA loads an entity that has a relationship (like an Order with a list of Items), it must decide: load the related data right now (EAGER) or wait until you actually ask for it (LAZY). This choice has a huge impact on performance.

Java Example

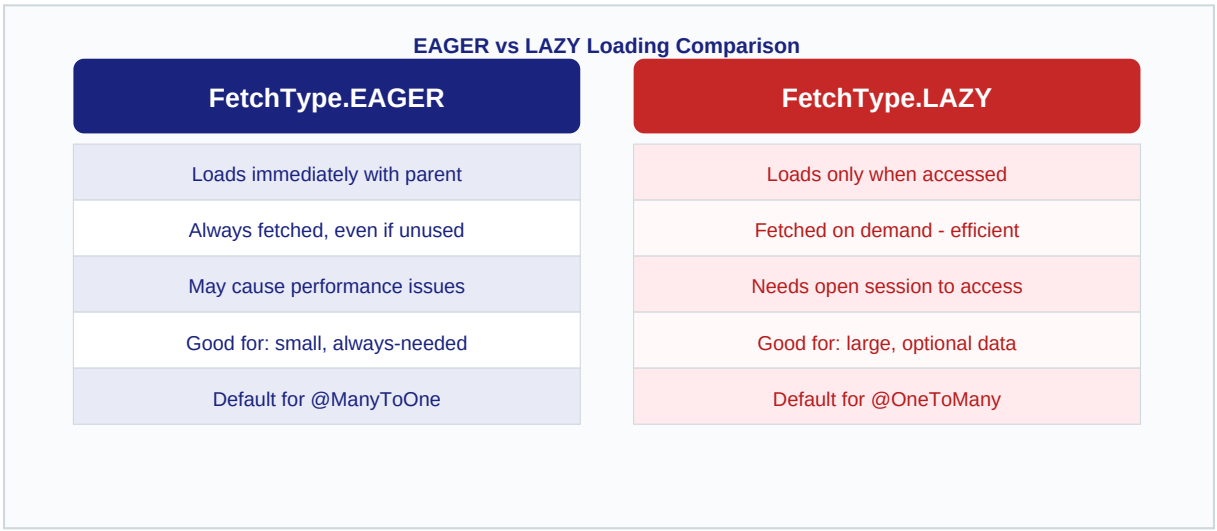
```
@Entity
public class Order {
    @Id Long id;
    String customerName;

    // EAGER: items loaded immediately when Order is loaded
    @OneToMany(fetch = FetchType.EAGER)
    List<Item> items;
}

@Entity
public class Order {
    @Id Long id;
    String customerName;

    // LAZY: items loaded only when order.getItems() is called
    @OneToMany(fetch = FetchType.LAZY)
    List<Item> items;
}
```

Diagram: EAGER vs LAZY Side-by-Side



Left: EAGER always fetches. Right: LAZY fetches on demand.

Key Rules

- FetchType.LAZY is the default for @OneToMany and @ManyToMany (collections).
- FetchType.EAGER is the default for @ManyToOne and @OneToOne (single objects).
- LAZY saves memory and time when the related data is not always needed.
- EAGER can cause over-fetching and unexpected SQL when loading many entities.
- Always prefer LAZY and fetch eagerly only when truly needed.

Q1. What does FetchType.EAGER mean?

The related entity or collection is loaded from the database immediately when the parent entity is loaded, in the same query or an additional query.

Q2. What is the default fetch type for @OneToMany?

FetchType.LAZY. Collections are not loaded until you call the getter on them.

Q3. What is the default fetch type for @ManyToOne?

FetchType.EAGER. Single associated entities are loaded immediately by default.

Q4. Why is LAZY generally preferred?

Because it avoids loading data you may never use, reducing memory usage and database load.

Q5. Can you change the fetch type at runtime?

Yes, using JOIN FETCH in JPQL or @EntityGraph you can override the default fetch plan for a specific query.

2 The N+1 Problem

1 query for list + N queries for each association

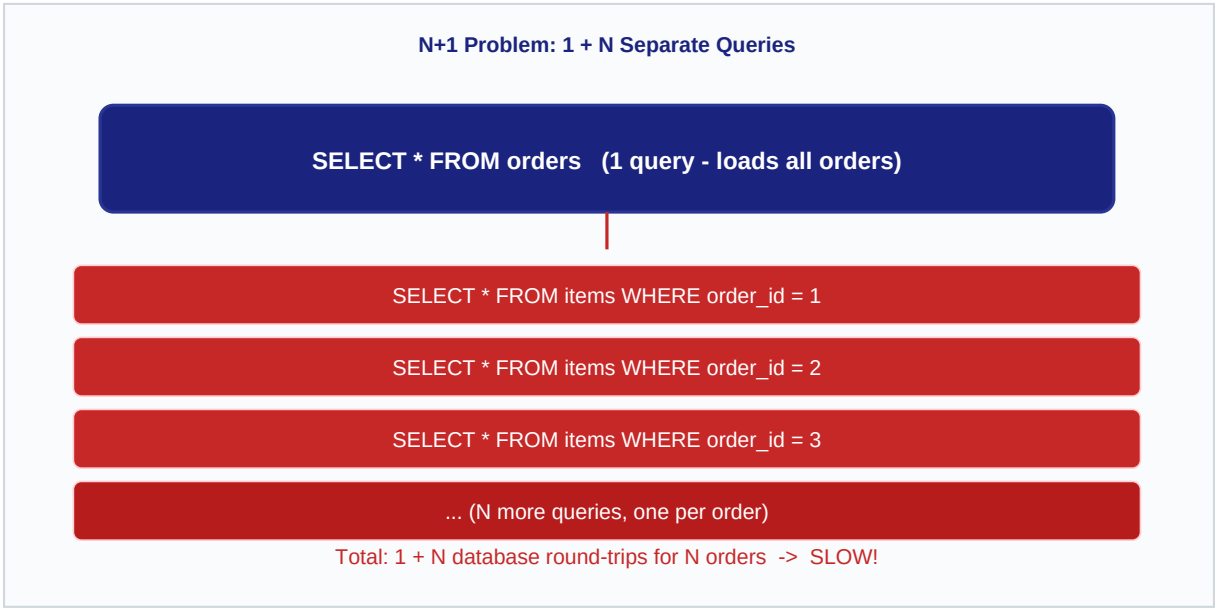
The N+1 problem is one of the most common JPA performance bugs. It happens when you load a list of N entities and then access a LAZY association on each one inside a loop. JPA fires 1 query to get the list, then N extra queries - one per entity - to load the association. For 100 orders that is 101 database queries!

Java Example (Buggy Code)

```
// 1 query: SELECT * FROM orders
List<Order> orders = em.createQuery(
    "SELECT o FROM Order o", Order.class
).getResultList();

for (Order o : orders) {
    // N queries: SELECT * FROM items WHERE order_id = ?
    // One extra query fires for EACH order!
    System.out.println(o.getItems().size());
}
// Total: 1 + N queries fired
```

Diagram: N+1 Query Pattern



1 large query loads the list; N small queries load each association separately.

Key Rules

- N+1 is a silent bug - the code looks correct but fires too many queries.
- It gets worse as data grows: 1000 orders = 1001 queries.
- Use SQL logging to detect N+1: look for repeated similar queries in the log.
- Spring Boot: set spring.jpa.show-sql=true to see all fired queries.
- The fix is to tell JPA to load associations in the same query (JOIN FETCH, @EntityGraph, @BatchSize).

Q1. What is the N+1 problem?

It is a performance issue where loading N entities triggers N additional queries to load their associations, resulting in 1 + N total database queries.

Q2. Why does N+1 happen with LAZY loading?

Because JPA does not load the association until you access it. Accessing it inside a loop triggers one query per entity.

Q3. How can you detect N+1 in a Spring Boot app?

Enable spring.jpa.show-sql=true and look for repeated SELECT statements with different IDs in the console.

Q4. Does N+1 only happen with collections?

No. It can also happen with @ManyToOne or @OneToOne associations if they are accessed in a loop.

Q5. Can N+1 happen with EAGER loading too?

Yes. If JPA uses separate SELECT queries instead of a JOIN for EAGER loading, you still get N+1.

3 Fix 1: JOIN FETCH in JPQL

Load parent and association in a single SQL query

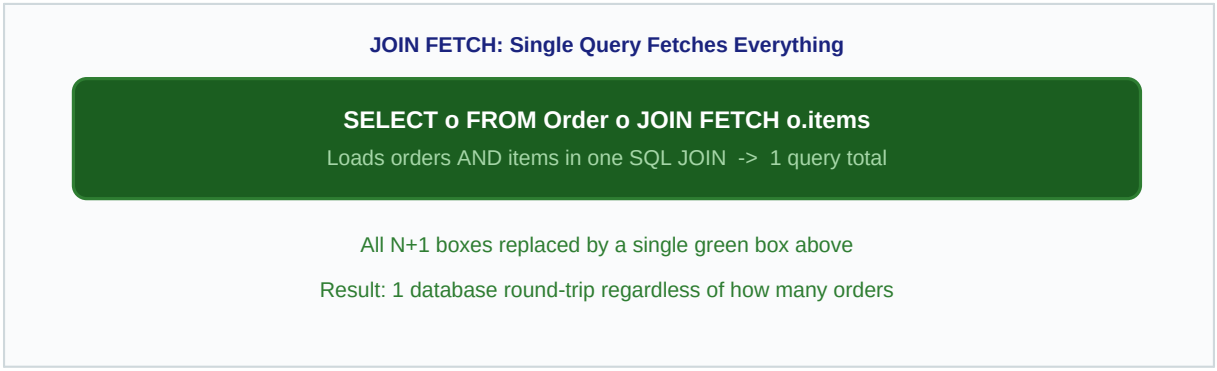
The simplest and most direct fix for N+1 is to use JOIN FETCH in your JPQL query. This tells Hibernate to generate a SQL JOIN that loads the parent entity and its association together in one round-trip to the database.

Java Example

```
// JOIN FETCH: loads orders AND items in ONE query
List<Order> orders = em.createQuery(
    "SELECT o FROM Order o JOIN FETCH o.items",
    Order.class
).getResultList();

for (Order o : orders) {
    // No extra query fires here - items already in memory
    System.out.println(o.getItems().size());
}
// Total: 1 query (a SQL JOIN)
```

Diagram: JOIN FETCH Replaces All N+1 Boxes



One JOIN FETCH query replaces the 1 + N query pattern entirely.

Key Rules

- JOIN FETCH is a JPQL keyword, not standard SQL - Hibernate translates it to a SQL JOIN.
- It overrides the LAZY fetch type for that specific query only.
- Use DISTINCT with JOIN FETCH on collections to avoid duplicate parent rows.
- You cannot use JOIN FETCH with setMaxResults (pagination) on collections - it causes a warning.
- For pagination with collections, prefer @BatchSize or @EntityGraph instead.

Q1. What does JOIN FETCH do in JPQL?

It tells Hibernate to generate a SQL JOIN query that loads the entity and its specified association together in a single database query.

Q2. Does JOIN FETCH change the fetch type permanently?

No. It only affects the specific query it is written in. The mapping annotation fetch type remains unchanged.

Q3. Why might JOIN FETCH cause duplicate results?

Because a SQL JOIN multiplies rows when there are multiple child records. Use DISTINCT in the JPQL to remove duplicates.

Q4. Can JOIN FETCH be used with pagination (setMaxResults)?

Not safely for collections. Hibernate will warn and fetch all rows in memory then paginate. Use @BatchSize or subqueries for pagination.

Q5. How does JOIN FETCH appear in the generated SQL?

Hibernate generates an INNER JOIN or LEFT OUTER JOIN (for LEFT JOIN FETCH) in the SQL, loading columns from both tables at once.

4 Fix 2: @BatchSize

Hibernate batches association queries to reduce round-trips

@BatchSize is a Hibernate-specific annotation that tells Hibernate to load associations in batches rather than one at a time. Instead of N queries, it fires ceil(N / batchSize) queries using SQL IN clauses. For 100 orders with @BatchSize(size=25) that is 4 queries instead of 100.

Java Example

```
@Entity
public class Order {
    @Id Long id;

    @OneToMany(fetch = FetchType.LAZY)
    @BatchSize(size = 25)   // Load items in batches of 25
    List<Item> items;
}

// Now when you loop 100 orders:
// Hibernate fires: SELECT ... WHERE order_id IN (1,2,...,25)
// Then:            SELECT ... WHERE order_id IN (26,27,...,50)
// Only 4 queries total instead of 100
```

Key Rules

- @BatchSize is Hibernate-specific, not standard JPA.
- It does not require changing your JPQL queries - just add the annotation.
- Typical batch sizes: 10, 25, or 50 depending on expected collection sizes.
- You can also set a global default: hibernate.default_batch_fetch_size in properties.
- @BatchSize works well with pagination since it does not use JOINs.

Q1. What does @BatchSize(size=25) do?

It tells Hibernate to load lazy collections in batches of 25 using SQL IN clauses, reducing N queries to approximately N/25 queries.

Q2. Is @BatchSize a standard JPA annotation?

No. It is a Hibernate-specific annotation from org.hibernate.annotations package.

Q3. How does Hibernate decide what to batch?

When you access a lazy collection, Hibernate also pre-loads collections for other already-loaded entities of the same type, up to the batch size.

Q4. Can @BatchSize be set globally?

Yes. Set hibernate.default_batch_fetch_size in your application.properties to apply it to all lazy associations.

Q5. When is @BatchSize better than JOIN FETCH?

When you need pagination, since JOIN FETCH does not work well with setMaxResults on collection associations.

5 Fix 3: @EntityGraph

JPA standard fetch plan - load associations on demand

@EntityGraph is a standard JPA 2.1 feature that lets you define which associations to fetch eagerly for a specific query, without changing the mapping annotations. It is the portable (non-Hibernate-specific) way to solve N+1 and is well supported in Spring Data JPA repositories.

Java Example

```
@Entity
@NamedEntityGraph(
    name = "Order.withItems",
    attributeNodes = @NamedAttributeNode("items")
)
public class Order {
    @Id Long id;

    @OneToMany(fetch = FetchType.LAZY)
    List<Item> items;
}

// In Spring Data JPA Repository:
@EntityGraph(attributePaths = {"items"})
List<Order> findAll();
// -> Loads orders + items in one query
```

Key Rules

- @EntityGraph is part of the JPA standard - works with any JPA provider.
- In Spring Data JPA, add @EntityGraph directly on the repository method.
- attributePaths lists which lazy associations to fetch eagerly for that query.
- Does not affect other methods or the mapping annotation - targeted and safe.
- For complex graphs with nested associations, use subgraphs in @NamedEntityGraph.

Q1. What is an EntityGraph?

It is a JPA feature that defines a fetch plan specifying which associations should be loaded eagerly for a particular query, overriding the default fetch type.

Q2. How is @EntityGraph used in Spring Data JPA?

Add the @EntityGraph annotation directly on a repository method with attributePaths listing the associations to fetch.

Q3. Is @EntityGraph Hibernate-specific?

No. It is a standard JPA 2.1 feature and works with any compliant JPA implementation.

Q4. What is the difference between @EntityGraph and JOIN FETCH?

@EntityGraph is a JPA standard and works well with Spring Data methods. JOIN FETCH requires writing JPQL manually. Both generate a similar SQL JOIN.

Q5. Can @EntityGraph load nested associations?

Yes. Use subgraphs in a @NamedEntityGraph or nest attributePaths like 'items.product' in Spring Data.

6 LazyInitializationException

Cause: accessing LAZY data after session closes

LazyInitializationException is thrown by Hibernate when you try to access a LAZY association after the JPA/Hibernate session (EntityManager) has already been closed. In web apps this often happens when a transaction ends in the service layer but the view or controller still tries to access lazy collections.

Java Example - Cause

```
@Transactional
public Order findOrder(Long id) {
    return orderRepo.findById(id).get();
    // Session closes here when method returns
}

// In controller (outside transaction):
Order o = service.findOrder(1L);
o.getItems().size();  // BOOM! LazyInitializationException
// Session is closed - cannot load items anymore
```

Fixes

```
// Fix 1: Use JOIN FETCH or @EntityGraph to load in transaction
@Transactional
public Order findOrderWithItems(Long id) {
    return orderRepo.findByIdWithItems(id); // uses @EntityGraph
}

// Fix 2: Use spring.jpa.open-in-view=false (recommended)
// and always fetch what you need inside the transaction

// Fix 3: Use a DTO to transfer data out of the transaction
public OrderDTO findOrderDTO(Long id) {
    Order o = orderRepo.findByIdWithItems(id);
    return new OrderDTO(o.getId(), o.getItems().size());
}
```

Key Rules

- LazyInitializationException means: session is closed but you tried to load lazy data.
- Root cause: accessing lazy association outside a @Transactional boundary.
- Never set spring.jpa.open-in-view=true as a permanent fix - it is an anti-pattern.
- Best fix: fetch required associations inside the transaction using JOIN FETCH or @EntityGraph.
- Use DTOs to transfer only the data you need across transaction boundaries.

Q1. What causes LazyInitializationException?

It is thrown when you access a LAZY association after the Hibernate session (EntityManager) has been closed. The proxy has no open session to execute the query.

Q2. How does open-in-view cause problems?

spring.jpa.open-in-view=true keeps the session open for the whole HTTP request, masking N+1 bugs and wasting database connections.

Q3. What is the recommended fix for LazyInitializationException?

Fetch all required associations inside the @Transactional service method using JOIN FETCH, @EntityGraph, or @BatchSize before the session closes.

Q4. Is using a DTO a good approach?

Yes. DTOs carry only the data needed by the caller and avoid passing Hibernate proxy objects across boundaries where the session may be closed.

Q5. Can you initialize a lazy collection manually inside a transaction?

Yes. Call Hibernate.initialize(entity.getCollection()) inside a transaction to force-load a lazy proxy before the session closes.

7 When to Use EAGER vs LAZY

Decision guide for real-world JPA projects

Choosing the right fetch strategy is a trade-off between convenience and performance. The general rule is: default to LAZY everywhere and switch to EAGER (or use JOIN FETCH) only for specific queries that need the data. This keeps your application fast and prevents unexpected database load.

Java Example - Recommended Pattern

```
@Entity
public class Order {
    @Id Long id;

    // LAZY by default - safe and performant
    @OneToMany(fetch = FetchType.LAZY)
    List<Item> items;

    // For @ManyToOne override EAGER to LAZY
    @ManyToOne(fetch = FetchType.LAZY)
    Customer customer;
}

// Fetch eagerly only when you need it:
@EntityGraph(attributePaths = {"items", "customer"})
Order findWithDetailsById(Long id);
```

Quick Decision Table

Situation	Recommended Strategy	Why
Large collection (orders, items, comments)	LAZY + JOIN FETCH or @EntityGraph when needed	Avoid loading thousands of rows automatically
Small lookup table always needed (e.g., Status, Category)	EAGER or JOIN FETCH	Small data, always used - safe to fetch always
@ManyToOne to a large entity	LAZY (override the default EAGER)	Prevents loading big parent entity unnecessarily
Paginated list with associations	LAZY + @BatchSize	JOIN FETCH breaks pagination on collections
REST API returning DTOs	LAZY + fetch inside transaction + map to DTO	Avoid lazy proxies escaping transaction boundary

Key Rules

- Default rule: use FetchType.LAZY for all relationships.
- Override to EAGER only for tiny, always-needed reference data.
- For @ManyToOne explicitly set FetchType.LAZY to override the JPA default.
- Use JOIN FETCH or @EntityGraph for specific queries that need associations.
- Profile your queries in production - what works in dev may be slow with real data.

Q1. What is the safest default fetch strategy?

FetchType.LAZY for all relationships. Fetch eagerly only for specific queries that need the data, using JOIN FETCH or @EntityGraph.

Q2. Should you always override @ManyToOne to LAZY?

Usually yes. The JPA default is EAGER for @ManyToOne, which loads the parent entity on every child load. Set LAZY unless the parent is always needed.

Q3. When is EAGER acceptable?

For small, mostly-static reference data that is always needed with the entity, such as a Status or Category with a handful of rows.

Q4. What fetch strategy works best with pagination?

@BatchSize is the safest for paginated collection queries. JOIN FETCH causes Hibernate to load all rows in memory before paginating.

Q5. How do you check if your fetch strategy is correct?

Enable SQL logging (spring.jpa.show-sql=true), run your use cases, and count the queries. Each use case should fire the minimum necessary queries.